

1. Discuss the significance of time and space complexity in algorithm design. Provide examples showing how trade-offs can occur between time and space.
2. Define the Big O, Big Θ (Theta), and Big Ω (Omega) notations. Illustrate their differences with graphical representation and suitable examples.
3. Describe the Divide and Conquer approach in detail. Illustrate with a dry run of Merge Sort on a sample array.
4. Differentiate between Greedy Algorithms and Dynamic Programming.
5. Analyze the time and space complexity of Quick Sort. What is the worst-case scenario and how can it be avoided?
6. Compare and contrast Heap Sort and Merge Sort in terms of performance, implementation, and stability.
7. Discuss linear-time sorting algorithms. How does Radix Sort manage to sort numbers faster than comparison-based sorting? Use a base-10 example to explain.
8. What is the principle behind Count Sort? Why is it not suitable for sorting floating-point or negative numbers?
9. Compare the time complexity of Linear and Binary Search. Under what conditions is Binary Search inefficient despite its logarithmic complexity?
10. Explain the concept of Order Statistics. How would you find the median in an unsorted array with linear time complexity?
11. Differentiate between single-dimensional and multi-dimensional arrays with examples.
12. What are sparse matrices? How are they represented efficiently?
13. Explain the concept of Stack. Also explain the PUSH and POP algorithms used in Stack with suitable example.
14. Implement a stack using arrays and linked lists. What are the advantages of each?
15. Differentiate between Prefix, Infix, and Postfix expressions. How do you convert from one form to another?
16. Write an algorithm to convert an infix notation to postfix notation using stack. Also apply the algorithm on the given infix notation:

$$A + (B - C) * (D + E \wedge F) / G$$
17. Write applications of stack.
18. Explain the concept of Queue using real life examples. Also illustrate Enqueue and Dequeue operations using in Queue.
19. Differentiate between various types of queues: simple queue, circular queue, dequeue, and priority queue. Where are each of these used in real-life applications?

20. Construct algorithms for insertion at the beginning, insertion at the given index, insertion after a given index and insertion at the end in case of simple linked list.
21. Construct algorithms for deletion of a node having specific key, deletion of node at the beginning and deletion of a node at the end in case of simple linked list.
22. Construct algorithms for insertion and deletion in a doubly linked list. Explain the handling of pointers in each case with diagrams.
23. How is a circular linked list different from a singly linked list? Write an algorithm to traverse a circular linked list safely.
24. What is recursion? Write a recursive algorithm for computing factorial.
25. Discuss the advantages and limitations of recursion with examples.
26. Develop recursive and iterative algorithms for calculating the GCD (Greatest Common Divisor) of two numbers. Compare their performance and call stack behaviour.
27. Differentiate between general tree and binary tree.
28. Write the algorithm for inserting and deleting a node from a Binary Search Tree. What are the cases to consider while deleting a node?
29. Discuss the three standard binary tree traversals: inorder, preorder, and postorder. Use the following tree for demonstration:


```

      100
     /  \
    50   120
   /  \   \
  24  65  180
      
```
30. Explain how trees can be used in expression evaluation. What is the significance of binary expression trees?